

PROFILE

Software Engineer with 14+ months of industry experience at Nielsen, building backend services, platform tooling, data pipeline orchestration, and CI/CD infrastructure across multiple projects. Backend-focused with full-stack capability, strong hands-on DevOps practices (Docker, Kubernetes, GitLab CI/CD), and a keen interest in microservices architecture and cloud-native deployment. Experienced in Agile/Scrum workflows with daily standups, Git branching strategies, code reviews, merge request approvals, and non-prod to prod release cycles.

EXPERIENCE

Software Development Engineer Intern — Nielsen

Unified Schema — Platform powering AskNielsen analytics across USA + 28 NGA countries

Mar 2025 – Present (14 mo) | Gurugram, India

- Built Python SDK client libraries wrapping internal Media Data Lake (MDL) and Trino (QaaS) APIs — adopted as the team's standard interface for schema registration, data ingestion, and query execution
- Designed data quality validation framework (pytest) with 5 validator types (syntax, uniqueness, non-null, allowed-values, cross-table referential integrity) — running via scheduled Airflow DAGs across 29 country datasets
- Built observability logging system (SQLAlchemy + PostgreSQL on RDS) tracking SQS events, worker DAG states, and validation results for full pipeline traceability
- Developed SQS event listener DAG processing incoming messages and triggering country-specific ETL worker pipelines
- Implemented DAG failure alerting via email and Google Chat webhooks with error tracebacks and task metadata

NTS — Kubernetes CronJob → Airflow Migration

- Migrated daily sync-check scripts from Kubernetes CronJobs to company Airflow using KubernetesPodOperator — added timezone mismatch filtering stage and enhanced alert details with Google Chat webhooks
- Set up GitLab CI/CD pipeline to build Docker images and push to GitLab Container Registry, pulled images in KPO DAGs, configured cross-account S3 access via IAM assume role, and secured Airflow variables with auto-masking for credentials

NTS — Big Data + Panel Data Pipeline Migration

- Migrated 6 weekly TV rating data files to a new format — modified legacy shell scripts and database config on a production Linux server to auto-detect and process new file patterns from an RSS-based data feed
- Built isolated test pipeline on production server, validated bulk loading of 200K+ records with zero errors across 4 final database tables, eliminating a manual weekly download-and-load workflow

PROJECTS

Microservices Chat Platform | Node.js, TypeScript, Express, Redis, RabbitMQ, Kubernetes

- 4-service system (Auth, User, Chat, API Gateway) with event-driven architecture — Auth publishes registration events via RabbitMQ, User Service consumes and syncs profiles, Chat Service caches user data from events, each service with its own database (MySQL, PostgreSQL, MongoDB)
- Shared common package with error handling, Zod request validation, JWT middleware, and inter-service auth via X-Internal-Token headers; Redis caching for conversations and search results
- Deployed on k3d Kubernetes cluster (3 nodes) with NGINX Ingress, ConfigMaps, Secrets, and multi-environment namespaces (dev/staging/prod)
- Full GitLab CI/CD pipeline — 4 stages (validate → build → docker → deploy), parallel service builds, commit SHA image tagging, multi-node K8s import, and automated health check verification

Patient Management System | Node.js/TypeScript + Python (complete duplication)

- 5-service microservices system (Auth, Patient, Billing, Analytics, API Gateway) — API Gateway validates JWT via Auth Service, Patient Service calls Billing via gRPC (Protocol Buffers) and publishes events to Kafka consumed by Analytics Service
- Each service with its own PostgreSQL database (Prisma), multi-stage Dockerfiles, Swagger/OpenAPI docs, and Vitest integration tests
- Full deployment: Docker Compose (9 containers), k3d Kubernetes cluster (3 nodes, Ingress, multi-replica deployments), and Nginx reverse proxy
- Complete Node.js implementation then fully duplicated in Python (FastAPI) — same architecture, same APIs, same deployment setup

Hushh Personal Agent | Next.js, TypeScript, PostgreSQL, Prisma, LangChain

- Built a full-stack AI-powered personal assistant with two AI personas, streaming chat integration (Ollama/OpenAI), encrypted user data vault, and per-persona consent-based data access — frontend developed using AI-assisted tooling (Kiro IDE)
- Implemented backend API routes, 12-model PostgreSQL schema with Prisma, user authentication, chat history, calendar, notifications, and admin escalation workflows

TECHNICAL SKILLS

Languages: Python, JavaScript/TypeScript, SQL, Java, C++

Architecture: Microservices, API Gateway, RabbitMQ, Kafka, SQS

Cloud & DevOps: AWS, Docker, Kubernetes, Helm, GitLab CI/CD

Frontend: React, Next.js (AI-assisted development)

Backend & APIs: FastAPI, Express.js, Next.js, REST, gRPC, GraphQL

Databases: PostgreSQL, ClickHouse, MongoDB, MySQL, OpenSearch

Orchestration: Apache Airflow (DAGs, Sensors, KPO), Event-driven ETL

Tools: Git, Linux, pytest, Prisma, SQLAlchemy, DBeaver

EDUCATION

Lovely Professional University — B.Tech CSE (2022–2026)

CGPA: 9.1/10 | NPTEL Cloud Computing Certified

The Air Force School (TAFS) — Senior Secondary (XII): 86% | Kendriya Vidyalaya AFS Rajokri — Secondary (X): 96.6%

CERTIFICATIONS

- Docker & Kubernetes: The Practical Guide — Udemy
- AWS Cloud Practitioner CLF-C02 — Udemy
- NPTEL Cloud Computing (Jul–Oct 2024)